یک الگوریتم، حالت سیستم را برای تعیین رخداد بن‌بست بررسی نموده و از الگوریتم دیگری برای رفع بن‌بست استفاده نماید (اگر به واقع بن‌بستی اتفاق افتاده باشد). این ملاحظات را در بخش ۸-۷ و بخش ۸-۸ بحث می‌کنیم.

با نداشتن الگوریتم‌هایی برای تشخیص و ترمیم از بن‌بست ممکن است به وضعیتی برسیم که سیستم در یک حالت بن‌بست قرار گرفته و حتی راهی برای تشخیص این که چه چیزی اتفاق افتاده است نیز در پیش رو نداریم. در این حالت، بن‌بست تشخیص داده نشده باعث تنزل کارایی سیستم می‌شود، چون منابع سیستم در اختیار نخ‌هایی است که نمی‌توانند اجرا شوند و نخ‌های بیشتری نیز باز هم با درخواست آن منابع وارد حالت بن‌بست می‌شوند. بالاخره، سیستم از کار می‌افتد و بایستی دوباره به صورت دستی راه‌اندازی شود.

اگرچه به نظر می‌رسد مشکل مسأله‌ی بن‌بست با این روش حل نمی‌شود، اما همان‌گونه که اشاره شد در بیشتر سیستم‌عامل‌ها استفاده دارد. هزینه به عنوان یک دغدغه‌ی مهم است. چشم‌پوشی از احتمال وقوع بن‌بست‌ها از رویکردهای دیگر ارزانتر است. چون در بسیاری از سیستم‌ها، بن‌بست‌ها خیلی کم پیش می‌آیند (بر فرض، یکبار در ماه)، هزینه‌ی اضافی روش‌های دیگر ممکن است به صرفه نباشد.

به علاوه، روش‌هایی که برای ترمیم از سایر شرایط عاقبت به‌خیری، همچون قفل زنده، استفاده می‌شوند ممکن است برای ترمیم از بن‌بست هم استفاده شوند. در برخی شرایط، یک سیستم از شکست عاقبت به‌خیری رنج می‌برد، اما یک حالت بن‌بست نیست. برای مثال، این وضعیت را می‌توان ناشی از یک نخ بلادرنگ دید که با بالاترین الویت اجرا می‌شود (یا هر نخی که با یک زمانبند بدون پس‌گیر اجرا دارد) و هرگز کنترل را به سیستم‌عامل بر نمی‌گرداند. در چنین شرایطی بایستی سیستم روش‌های ترمیم دستی داشته باشد و حتی می‌تواند از این تکنیک‌ها به سادگی برای رفع بن‌بست نیز استفاده کند.

۸-۵ پیشگیری از بن‌بست

همان‌گونه که در بخش ۸-۳-۱ متذکر شدیم، برقراری هر چهار شرط برای وقوع بن‌بست ضرورت دارد. اگر به این اطمینان برسیم که حداقل یکی از این شرط‌ها نمی‌تواند برقرار شود می‌توانیم از بن‌بست به دور باشیم. در اینجا، مطابق این رویکرد، هر یک از این چهار شرط لازم را مورد بررسی قرار می‌دهیم.

۸-۵-۱ انحصار متقابل

شرط انحصار متقابل بایستی برقرار شود. یعنی، حداقل یک منبع می‌بایست اشتراک‌ناپذیر باشد. منابع اشتراک‌پذیر، نیازی به دسترسی انحصاری نداشته و لذا نمی‌توانند در یک بن‌بست واقع شوند. فایل‌های فقط‌خواندنی مثال خوبی برای منابع اشتراک‌پذیر به حساب می‌آیند. اگر چندین نخ بخواهند یک فایل فقط‌خواندنی را همزمان باز کنند، دسترسی همزمان فایل قابل اعطا می‌باشد. یک نخ هرگز نیازی به انتظار منبع اشتراک‌پذیر نمی‌بیند. با این وجود، در حالت کلی نمی‌توان با نقض شرط انحصار متقابل از بن‌بست‌ها پیشگیری کرد، چون برخی از منابع ذاتاً

قابل اشتراک نیستند. برای مثال، یک قفل انحصار متقابل نمی‌تواند به صورت همزمان توسط نخ‌های متعدد به اشتراک گرفته شود.

۸-۵-۲ گرفتن و منتظر ماندن

برای این که تضمین کنیم هرگز شرط گرفتن و منتظر ماندن در سیستم پیش نمی‌آید، بایستی اطمینان بدهیم هر بار که نخی منبعی را درخواست می‌نماید، هیچ منبع دیگری در اختیار ندارد. یک پروتکل که در اینجا می‌توانیم استفاده کنیم این است که اجبار کنیم هر نخ، قبل از آغاز اجرایش تمامی منابع خود را درخواست کرده و در اختیار بگیرد. البته، این امر برای اکثر برنامه‌ها به‌دلیل طبیعت پویای درخواست منابع غیرعملی است.

پروتکل دیگری وجود دارد که تنها در صورتی به یک نخ اجازه می‌دهد منابعی را درخواست کند که منبعی در اختیار نداشته باشد. یک نخ ممکن است منابعی را درخواست داده و از آنها استفاده کند. با این همه، پیش از این که بتواند منابع دیگری را بدست آورد، بایستی تمامی منابع فعلی در اختیار خود را رها سازد.

این پروتکل‌ها دو عیب مهم دارند. اولاً، بهره‌وری منبع می‌تواند پایین بیاید، چون ممکن است منابع به رغم تخصیص برای دوره‌ای طولانی استفاده نداشته باشند. برای مثال، یک نخ ممکن است یک قفل انحصار متقابل را برای اجرای کامل خودش در اختیار بگیرد، در عین حالی که تنها برای یک فاصله‌ی کوتاهی لازم خواهد داشت. ثانیاً، قحطی‌زدگی امکان‌پذیر است. نخی که نیازمند چندین منبع همگانی می‌باشد شاید تا ابد منتظر بماند، چون حداقل یکی از منابعی که لازم دارد همیشه در اختیار نخ‌های دیگری قرار خواهد داشت.

۸-۵-۳ نبود پس‌گیری

سومین شرط لازم برای وقوع بن‌بست‌ها، نداشتن پس‌گیری برای منابعی است که پیشتر تخصیص یافته‌اند. برای اطمینان از این که این شرط برقرار نمی‌شود از این پروتکل استفاده می‌کنیم. اگر نخی منابعی را در اختیار گرفته و منبع دیگری را درخواست می‌نماید که بلافاصله قابل اعطا نیست (یعنی، نخ بایستی به ناچار منتظر شود)، در این صورت تمامی منابع در اختیار نخ پس‌گرفته می‌شوند. به عبارت دیگر، این منابع به صورت ضمنی آزاد می‌شوند. منابع پس‌گرفته شده به لیست منابعی که نخ، منتظر آنهاست افزوده می‌شوند. نخ فقط در صورتی می‌تواند کار خود را دوباره شروع کند که منابع قبلی را در کنار منابع درخواستی جدید خود به‌دست آورد.

در روشی دیگر، اگر یک نخ منابعی را درخواست کند، ابتدا بررسی می‌کنیم که آیا آن منابع در دسترس هستند. اگر موجود بودند، آنها را اعطا می‌کنیم. اگر در دسترس نبودند، بررسی می‌کنیم تا ببینیم آیا به نخ‌هایی تخصیص یافته‌اند که آنها هم منتظر منابع اضافه‌تری هستند. اگر چنین بود، منابع مطلوب را از نخ منتظر پس‌گرفته و به نخی که آنها را درخواست کرده، اعطا می‌کنیم. اما، اگر منابع در دسترس نباشند و در اختیار نخ منتظری هم نبودند، نخ درخواست‌کننده ناگزیر منتظر می‌ماند. در حالی که نخ منتظر می‌شود، امکان دارد برخی از منابع در اختیار آن با درخواست

نخی دیگر پس‌گرفته شوند. به این ترتیب، نخ فقط در صورتی کار خود را دوباره شروع خواهد کرد که منابع جدید خود و هر منبعی را که در زمان انتظار پس داده است در اختیار بگیرد.

این پروتکل غالباً برای منابعی همچون ثبات‌های پردازنده و تراکنش‌های پایگاه‌داده اِعمال می‌شود. آن نمی‌تواند در حالت کلی برای منابعی همچون قفل‌های انحصار متقابل و سمافورها اِعمال شود، دقیقاً نوع منابعی که بن‌بست بسیار زیاد اتفاق می‌افتد.

۸-۵-۴ انتظار چرخشی

سه گزینه‌ای که تابحال برای پیشگیری از بن‌بست مطرح شده عموماً در اکثر شرایط غیرعملی هستند. با این وجود، چهارمین و آخرین شرط وقوع بن‌بست‌ها—شرط انتظار چرخشی—یک فرصتی برای یک راه‌حل عملی با نامعتبرساختن یکی از شرایط لازم ارائه می‌دهد. یک روش تضمین عدم برقراری این شرط آن است که روی تمامی انواع منابع، یک ترتیب کلی اعمال شود و نخ‌ها ملزم شوند که درخواست منابع خود را به ترتیب افزایش شماره انجام بدهند.

برای روشن‌شدن مطلب، مجموعه‌ی $R = \{R_1, R_2, \ldots, R_m\}$ را مجموعه‌ی انواع منابع اختیار می‌کنیم. برای هر نوع منبع، شماره‌ای یکتا نسبت می‌دهیم که به ما امکان مقایسه‌ی منابع به صورت دو به دو و تعیین ترتیب پس و پیش آنها را فراهم می‌کند. تابع یک‌به‌یک $F: R \rightarrow N$ را به‌صورت رسمی[1] تعریف می‌کنیم که در آن، N مجموعه‌ی اعداد طبیعی است. می‌توانیم این طرح را در یک برنامه‌ی کاربردی با گذاشتن یک ترتیبی مابین تمامی اشیاء همگامی در سیستم انجام دهیم. برای مثال، ترتیب قفل در برنامه‌ی نخ پوزیکس نشان داده شده در شکل ۸-۱ می‌تواند به‌صورت زیر باشد:

```
F(first_mutex) = 1
F(second_mutex) = 5
```

اکنون می‌توانیم از این پروتکل برای پیشگیری بن‌بست‌ها استفاده کنیم: هر نخ می‌تواند منابع را تنها به ترتیب افزایش شماره درخواست کند. یعنی، یک نخ در ابتدا می‌تواند نمونه‌ای از یک منبع—برفرض، R_i—را درخواست نماید. بعد از آن، نخ می‌تواند نمونه‌ای از منبع R_j را درخواست کند اگر و تنها اگر $F(R_j) > F(R_i)$ باشد. برای مثال، با استفاده از تابع تعریف شده در بالا، یک نخ که می‌خواهد همزمان از هر دوی first_mutex و second_mutex استفاده کند، ابتدا بایستی درخواستی برای first_mutex و سپس درخواستی برای second_mutex انجام بدهد. در روشی دیگر، می‌توان الزام کرد که هر بار، نخ نمونه‌ای از منبع R_j را درخواست می‌نماید، تمامی منابع R_i را که $F(R_i) \geq F(R_j)$ می‌باشد، پیش از درخواست رها کند. همچنین قابل توجه است که اگر نمونه‌های متعددی از یک نوع منبع مورد نیاز باشند، بایستی برای تمامی آنها یک درخواست منفرد صادر شود.

اگر این دو پروتکل مورد استفاده قرار بگیرند، در این صورت شرط انتظار چرخشی نمی‌تواند برقرار شود. برای نشان دادن درستی این موضوع فرض کنیم که یک انتظار چرخشی وجود دارد (فرض خلف). اگر مجموعه‌ی $\{T_0, T_1, \ldots, T_n\}$ نخ‌هایی باشند که در انتظار چرخشی

1. Formally

قرار دارند، T_i منتظر منبع R_i می‌باشد که T_{i+1} در اختیار دارد. (برای محاسبه‌ی اندیس‌ها از حساب مدول استفاده می‌شود، لذا نخ T_n منتظر منبع R_n است که در اختیار P_0 می‌باشد.) در این صورت، چون نخ T_{i+1} منبع R_i را در اختیار دارد و منبع R_{i+1} را درخواست داده است، بایستی برای تمامی iها رابطه‌ی $F(R_i) < F(R_{i+1})$ برقرار باشد. اما این شرط به معنی $F(R_0) < F(R_1) < \ldots < F(R_n) < F(R_0)$ است. طبق خاصیت تراگذری به رابطه‌ی $F(R_0) < F(R_0)$ می‌رسیم که ناممکن است. بنابراین، هیچ‌گونه انتظار چرخشی نمی‌تواند برقرار باشد.

به خاطر داشته باشید که اتخاذ یک ترتیب یا سلسله‌مراتب به خودی خود نمی‌تواند از بن‌بست پیشگیری کند. نویسندگان برنامه‌های کاربردی نیز می‌بایست این ترتیب را رعایت نمایند. با این وجود، برپایی یک ترتیب قفل، مخصوصاً در سیستمی با صدها—یا هزاران—قفل می‌تواند دشوار باشد. برای لحاظ این چالش، بسیاری از برنامه‌نویسان جاوا راهبرد استفاده از متد System.identityHashCode(Object) (که مقدار کد درهم پیش‌فرض پارامتر Object ارسال شده به آن را بر می‌گرداند) را به‌عنوان تابعی برای ترتیب‌دهی کسب قفل انتخاب کرده‌اند.

همچنین قابل توجه است که اگر قفل‌ها بتوانند به صورت پویا کسب شوند، اِعمال یک ترتیب قفل نمی‌تواند ضامن پیشگیری از بن‌بست باشد. برای مثال، فرض کنید که تابعی داریم که انتقال وجه بین دو حساب بانکی را انجام می‌دهد. جهت پیشگیری از پیشامد یک شرط مسابقه، هر حساب بانکی یک قفل انحصار متقابل مرتبط دارد که با تابع ()get_lock آن‌گونه که در شکل ۸–۷ نشان داده شده کسب می‌شود. اگر تابع ()transaction توسط دو نخ با دو حساب بانکی متفاوت به صورت پس و پیش احضار شود، امکان بن‌بست وجود دارد. یعنی، یک نخ امکان دارد که تابع را به صورت زیر احضار نماید:

```
transaction(checking_account, savings_account, 25.0);
```

و نخ دیگر هم می‌تواند احضار زیر را انجام بدهد:

```
transaction(savings_account, checking_account, 50.0);
```

رفع این وضعیت را به عنوان یک تمرین برای دانش‌جویان واگذار می‌کنیم.

```
void transaction(Account from, Account to, double amount)
{
        mutex lock1, lock2;
        lock1 = get_lock(from);
        lock2 = get_lock(to);
        acquire(lock1);
        acquire(lock2);
        withdraw(from, amount);
        deposit(to, amount);
        release(lock2);
        release(lock1);
}
```

شکل ۸–۷ مثال بن‌بست با ترتیب قفل.